

API_REFERENCE

Challenges API Reference

Package challenge

Import: digital.vasic.challenges/pkg/challenge

Core types and interfaces for challenge definition.

Interface Challenge

```
type Challenge interface {
    ID() string
    Configure(config Config) error
    Validate() error
    Execute(ctx context.Context) error
    Cleanup(ctx context.Context) error
    Dependencies() []string
    Tags() []string
}
```

Struct Config

```
type Config struct {
    ID          string
    Name        string
    Description  string
    Tags        []string
    Dependencies []string
    Timeout     time.Duration
    RetryPolicy *RetryPolicy
    Assertions  map[string]interface{}
    Environment map[string]string
}
```

Struct Result

```
type Result struct {
    ChallengeID string
    Status      Status
    StartTime   time.Time
    EndTime     time.Time
    Duration    time.Duration
    Error       error
    Output      string
    Assertions  map[string]AssertionResult
}
```

Type Status

```
type Status string

const (
    StatusPending Status = "pending"
    StatusRunning Status = "running"
    StatusPassed  Status = "passed"
    StatusFailed  Status = "failed"
    StatusSkipped Status = "skipped"
    StatusError   Status = "error"
)
```

Package registry

Import: digital.vasic.challenges/pkg/registry

Challenge registration and dependency management.

Interface Registry

```
type Registry interface {
    Register(challenge Challenge) error
    Get(id string) (Challenge, error)
    GetAll() []Challenge
    GetByTag(tag string) []Challenge
    GetExecutionOrder() ([]string, error)
}
```

Function New

```
func New() Registry
```

Creates a new challenge registry.

Example:

```
reg := registry.New()
reg.Register(&MyChallenge{id: "test-1"})
order, _ := reg.GetExecutionOrder() // Topologically sorted
```

Package runner

Import: digital.vasic.challenges/pkg/runner

Challenge execution engine.

Interface Runner

```
type Runner interface {
    Run(ctx context.Context, challengeID string) (*Result, error)
    RunAll(ctx context.Context) ([]*Result, error)
    RunSequence(ctx context.Context, challengeIDs []string)
        ([]*Result, error)
    RunParallel(ctx context.Context, challengeIDs []string)
        ([]*Result, error)
}
```

Function New

```
func New(registry registry.Registry, opts ...Option) Runner
```

Options:

- WithParallelism(int) -- Max concurrent challenges
- WithTimeout(time.Duration) -- Global timeout
- WithLogger(logging.Logger) -- Logger instance
- WithMonitor(monitor.Monitor) -- Live monitoring

Example:

```
runner := runner.New(reg,
    runner.WithParallelism(5),
    runner.WithTimeout(30*time.Second),
)

results, _ := runner.RunAll(ctx)
```

Package assertion

Import: digital.vasic.challenges/pkg/assertion

Assertion evaluation engine.

Interface Engine

```
type Engine interface {  
    Evaluate(ctx context.Context, assertions  
        map[string]interface{}) error  
    RegisterEvaluator(name string, evaluator Evaluator) error  
}
```

Type Evaluator

```
type Evaluator func(value interface{}, expected interface{})  
    (bool, error)
```

Function NewEngine

```
func NewEngine() Engine
```

Creates assertion engine with 16 built-in evaluators.

Built-in Evaluators:

- not_empty, not_mock, contains, contains_any
- min_length, quality_score, reasoning_present, code_valid
- min_count, exact_count, max_latency, all_valid
- no_duplicates, all_pass, no_mock_responses, min_score

Example:

```
engine := assertion.NewEngine()  
err := engine.Evaluate(ctx, map[string]interface{}{  
    "response": {  
        "not_empty": true,  
        "min_length": 10,  
        "contains": "success",  
    },  
    "latency": {  
        "max_latency": 1000, // milliseconds  
    },  
})
```

Package report

Import: digital.vasic.challenges/pkg/report

Report generation in multiple formats.

Interface Reporter

```
type Reporter interface {  
    Generate(results []*Result) string  
    GenerateToFile(results []*Result, filepath string) error  
}
```

Function NewMarkdownReporter

```
func NewMarkdownReporter() Reporter
```

Function NewJSONReporter

```
func NewJSONReporter() Reporter
```

Function NewHTMLReporter

```
func NewHTMLReporter() Reporter
```

Example:

```
reporter := report.NewMarkdownReporter()  
markdown := reporter.Generate(results)  
fmt.Println(markdown)  
  
// Or save to file  
reporter.GenerateToFile(results, "report.md")
```

Package plugin

Import: digital.vasic.challenges/pkg/plugin

Plugin system for custom challenge types.

Interface Plugin

```
type Plugin interface {  
    Name() string  
    Version() string  
    Init(config map[string]interface{}) error  
    CreateChallenge(id string, config challenge.Config)  
        (challenge.Challenge, error)  
    Shutdown() error  
}
```

Function RegisterPlugin

```
func RegisterPlugin(plugin Plugin) error
```

Example:

```
type CustomPlugin struct {}
```

```
func (p *CustomPlugin) CreateChallenge(id string, config
    challenge.Config) (challenge.Challenge, error) {
    return &CustomChallenge{id: id, config: config}, nil
}
```

```
plugin.RegisterPlugin(&CustomPlugin{})
```

Complete Usage Example

```
package main
```

```
import (
    "context"
    "fmt"
    "time"
```

```
    "digital.vasic.challenges/pkg/challenge"
    "digital.vasic.challenges/pkg/registry"
    "digital.vasic.challenges/pkg/runner"
    "digital.vasic.challenges/pkg/assertion"
    "digital.vasic.challenges/pkg/report"
)
```

```
// Custom challenge implementation
```

```
type APITestChallenge struct {
    challenge.BaseChallenge
    endpoint string
}
```

```
func (c *APITestChallenge) Execute(ctx context.Context) error {
    // Make API call
    resp, err := http.Get(c.endpoint)
    if err != nil {
        return err
    }
    defer resp.Body.Close()

    body, _ := io.ReadAll(resp.Body)

    // Evaluate assertions
    return c.AssertionEngine.Evaluate(ctx, map[string]interface{}{
        "status_code": resp.StatusCode,
```

```

        "response_body": string(body),
        "assertions": map[string]interface{}{
            "not_empty": true,
            "contains": "success",
            "min_length": 10,
        },
    })
}

func main() {
    ctx := context.Background()

    // Create registry
    reg := registry.New()

    // Register challenges
    reg.Register(&APITestChallenge{
        BaseChallenge: challenge.BaseChallenge{
            ChallengeID: "api-test-1",
        },
        endpoint: "http://localhost:8080/api/health",
    })

    // Create runner
    run := runner.New(reg,
        runner.WithParallelism(5),
        runner.WithTimeout(30*time.Second),
    )

    // Execute all challenges
    results, err := run.RunAll(ctx)
    if err != nil {
        fmt.Println("Execution error:", err)
        return
    }

    // Generate reports
    mdReporter := report.NewMarkdownReporter()
    markdown := mdReporter.Generate(results)
    fmt.Println(markdown)

    jsonReporter := report.NewJSONReporter()
    jsonReporter.GenerateToFile(results, "results.json")

    // Print summary
    passed := 0
    for _, result := range results {
        if result.Status == challenge.StatusPassed {
            passed++
        }
    }
}

```

```
    fmt.Printf("\nPassed: %d/%d\n", passed, len(results))  
}
```

Last Updated: February 10, 2026 **Version:** 1.0.0 **Total API**
Methods: 40+ **Status:**  Complete